

LensArt: Art Recognition and Discovery System

Karthik Javvaji
 MS Data Science
 Regis University, Denver, CO, USA
 kjavvaji@regis.edu

Abstract

LensArt is a Art Recognition and Discovery System which works on a deep learning system that takes the uploaded image of any painting and provides more context to it like the artist name, a simple amount of information regarding the artist and the historical context behind the painting giving depth to the work than just a title to the image, which is derived from Wikipedia and information from the Metropolitan Museum of Art working as a fallback and any similar works based on the artistic style. In this system the architecture uses a combination of two models which are namely a ResNet50 model transfer trained on the 10 classes ArtBench-10 dataset for art style classification, and a pretrained OpenCLIP ViT-B/32 model operating in zero shot method for identifying the artist and embedding the image for vector-based similarity search. The final output is a Flask web application which allows the user to upload any image taken of a painting and get more context to it in terms of the artist, the art style used and even the historical significance of it, currently running entirely on the local CPU and can provide output under 1 to 2 seconds after initial model warmup.

I. INTRODUCTION

Art exists everywhere, mainly in places like galleries, museums, social media and even on books. Any casual user cannot fully immerse themselves as we lack expertise in distinguishing the various styles, importance of the historical background behind that painting as most of mainstream applications like Google Lens give the user only similar image matches but lack at providing context to the artwork and artist's background. LensArt bridges that gap in knowledge for all of us who lack expertise in the field of art. Our target audience for this tool are art enthusiasts, museum visitors and students who would want to get more context or discover their own preference in art.

The system uses a combination of two Convolutional Neural Networks (CNN) that efficiently work on tasks in the model. The final system uses a transfer trained ResNet50 model to classify art styles while it employs a CLIP model to identify the artist's name and show visual similarities based on cosine similarity search.

II. PROBLEM STATEMENT

The process of determining who painted an artwork based just by looking at the image needs expertise. Furthermore, determining an artistic style and the movement demands immense knowledge in both art history and keen observational expertise which needs great amount of formal education in arts. The mainstream image search system available display matching pixels or colors in similar images but fail to provide the story behind the artwork such like the Renaissance period lasting from 1400 to 1550 brought a new direction in the art movement which shifted from the medieval artistic traditions to a more realistic style that used scientific methods of representation and in the Baroque period lasting from 1600 to 1750 introduced an strong theatrical emotions with usage of deep colors and effects which churches used to display their authority over the society. No existing application currently combines both the artist, style identification while providing a similarity search feature with biographical information as context to the painting [1], [2].

III. PROJECT OBJECTIVES

This complete system was created to achieve six major goals. First, to develop a working end-to-end art recognition system pipeline that will accept the uploaded image of a painting and return the artist's details with a confidence score, the art stylistic movement, similar works based on the style or artist, and a short artist biography giving context to it. Second, to use a transfer trained ResNet50 tuned on ArtBench-10 for style classification [3]. Third, using OpenCLIP in zero-shot mode for artist identification [4], [5]. Fourth, to prepare image embeddings for similarity search using OpenCLIP. Fifth, to use Rest API to pull relevant biographical information from Wikipedia and Metropolitan Museum of Art APIs for providing more context and background [6]. Sixth, to provide the complete system that works on the Flask web application using a virtual environment setup that runs on CPU.

IV. INITIAL PLAN

The original proposal was intended to do a fully supervised pipeline and a dataset made from WikiArt scrape of approximately 22,000 paintings including 50 artists profiles and 27 stylistic movements [7]. We used a combination of two models which included ResNet50 transfer learning baseline and primary Vision Transformer ViT-B/16 model with augmentation and layer wise learning-rate and AMP mixed-precision training [8] operation. The system used a frozen CLIP ViT-B/32 image encoder which worked together with a FAISS index to solve visual similarity problems. The project repository contains the complete code for both the scraper and training scripts.

V. CONSTRAINTS ENCOUNTERED

A. *Compute Budget*

It requires approximately 14 hours to fine-tune a ViT-B/16 model using 22,000 images on a GPU; without GPU access, CPU-only training required more than seven days for each experiment which made it really difficult to train through multiple changes during the project.

B. *Scrape Reliability and Dataset Assembly*

WikiArt's internal endpoint operates under rate limits which lead to recurrent HTTP 503 errors [7]. The process of assembling the complete dataset needs 24 to 48 hours of scraping work and it creates issues related to copyright redistribution. The scrape reached its operational capacity because of technical restrictions which recurrently kept stopping the algorithm from working.

C. *Deliverable Shape*

The deliverable must run on a reviewer's laptop. The project needs to deliver CPU-only inference that takes less than two seconds because a GPU-trained heavyweight model cannot be delivered to users.

D. *Decision: Hybrid Pipeline (ResNet50 + CLIP)*

The project pivoted to a hybrid system which combines ResNet50 for style classification of the concise balanced ArtBench-10 corpus [3] and CLIP ViT-B/32 for artist identification which requires no prior training because as there was no available labeled artist data openly [4], [5]. The same CLIP embedding functions as the basis for visual similarity search.

VI. DATASET

The art style training process uses ArtBench-10 which provides 60,000 painting images that show 10 equal art styles which include art styles like nouveau, baroque, expressionism, impressionism, post impressionism, realism, renaissance, romanticism, surrealism and ukiyo. The process used PIL to delete corrupted files while torchvision handled image resizing to 224×224 dimensions and created PyTorch tensors. The style classes include primary art movements from Western and East Asian traditions which cover Renaissance art from 1400 to 1550 Baroque art from 1600 to 1750 Impressionism art from 1865 to 1885 and Expressionism/Surrealism art from the Cubist time period [9].

CLIP require no training data as it is already pre-trained on 400M images. The reference painting collection located in `sample_images/` gets converted into 512-dimensional feature vectors which get stored as `index_cache/embeddings.npy` to enable similarity search during query execution.

VII. CNN ARCHITECTURE — RESNET50

The art style classifier uses a ResNet50 [10] with ImageNet-pretrained weights. The skip connections of ResNet50 solve the vanishing gradient issue which allows the network to maintain stable training throughout its 50-plus layers. The architecture processes a 224×224 RGB input through an initial 7×7 conv layer, max-pooling, and four residual stages containing 3, 4, 6, and 3 bottleneck blocks, producing a 2048-dimensional feature vector via global average pooling. For LensArt, the original 1000-way head is replaced with a 10-way linear layer matching the ArtBench-10 classes [3], [11]. The complete description of the layers appears in Table I.

TABLE I
RESNET50 LAYER BREAKDOWN (LENSART CONFIGURATION)

Layer / Block	Output Shape	Params	Notes
Input	$3 \times 224 \times 224$	—	RGB, ImageNet mean/std
Conv1 (7×7 , stride 2)	$64 \times 112 \times 112$	9,408	BatchNorm + ReLU
MaxPool (3×3 , s2)	$64 \times 56 \times 56$	—	
Layer1 — $3 \times$ Block	$256 \times 56 \times 56$	215,808	Residual blocks
Layer2 — $4 \times$ Block	$512 \times 28 \times 28$	1,219,584	Stride 2 at first
Layer3 — $6 \times$ Block	$1024 \times 14 \times 14$	7,098,368	Stride 2 at first
Layer4 — $3 \times$ Block	$2048 \times 7 \times 7$	14,964,736	Stride 2 at first
AdaptiveAvgPool	$2048 \times 1 \times 1$	—	Global avg pool
Flatten	2048	—	
FC head (10-way)	10	20,490	Replaces 1000-way head
Total	—	$\sim 23.5\text{M}$	$\sim 95\text{ MB}$ on disk

VIII. METHODOLOGY

A. ResNet50 — Supervised Style Classification

The training process of ResNet50 starts with head-only training which keeps the backbone network unchanged before moving to full network training with a learning rate that is 10 times lower. The system uses cross-entropy loss with label smoothing 0.05 together with the Adam optimizer [12] and a cosine-annealing LR schedule that runs for 25 CPU training epochs. The model checkpoint stored at `models/resnet_model.pth` achieves a validation accuracy of approximately 61.7 percent.

B. CLIP — Zero-Shot Artist Classification

The CLIP system uses CLIP ViT-B/32 [4], [5] to transform artist prompts which use the format “a painting by {artist}” into a precomputed text-feature matrix that contains 41 artists and 512 text features. The system processes image queries by calculating the dot product of the image embedding with the matrix and using softmax with temperature 30 to generate a probability distribution which identifies the top- K artists.

C. Visual Similarity Search

Paintings used as references undergo encoding into an $N \times 512$ CLIP embedding matrix which is then stored on disk as a permanent record. The system retrieves five closest neighbors after excluding matching items through cosine similarity calculations which operate on all matrix rows.

D. Enrichment

The system queries Wikipedia through its REST API [6] using the main artist prediction while using the Met Museum Open Access API as an alternative method. The two clients operate with a silent error mechanism which causes UI elements to remain operational during their 10-second timeout period.

IX. SYSTEM ARCHITECTURE

The pipeline flows through four sequential layers shown in Fig. 1: (1) Image input via Flask upload; (2) Hybrid ML backend — ResNet50 for style, CLIP for artist and similarity; (3) Biographical enrichment via Wikipedia with Met Museum fallback; (4) Integrated results rendered in the Flask UI. The system consists of Python modules which allow users to change individual components without affecting other parts of the system.

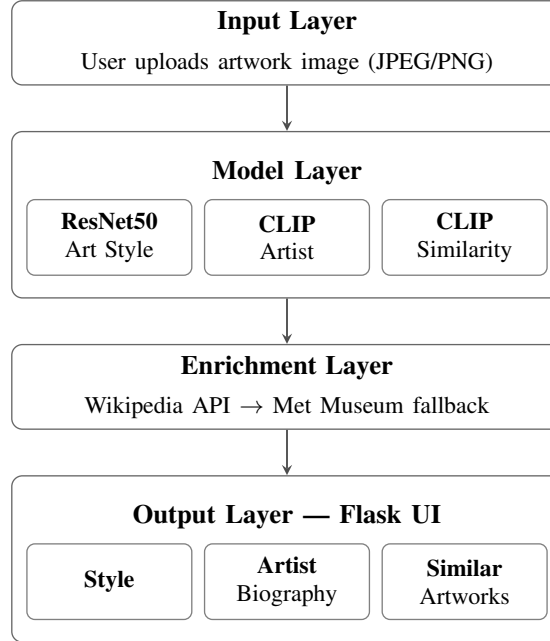


Fig. 1. LensArt system architecture — four sequential processing layers.

X. SYSTEM IMPLEMENTATION

The system requires Python version 3.10 or higher for its operation through the `lensart` package. The `app.py` file connects Flask routes while `resnet_classifier.py` accesses the ResNet50 checkpoint and `clip_model.py` provides CLIP ViT-B/32 functionality and `similarity.py` handles embedding operations and `enrichment.py` manages both APIs and `config.py` stores the artist and style universes. The `setup_venv.bat` file creates an isolated Python environment with all required dependencies which occupies approximately 600 megabytes of space and includes CPU-only PyTorch. The `run.bat` file initiates Flask while it directs users to `http://localhost:5000`.

The landing page presents an upload interface together with five pre-installed sample artworks according to Fig. 2. Each result section displays its corresponding model or API identifier which enables users to see how the entire process operates.

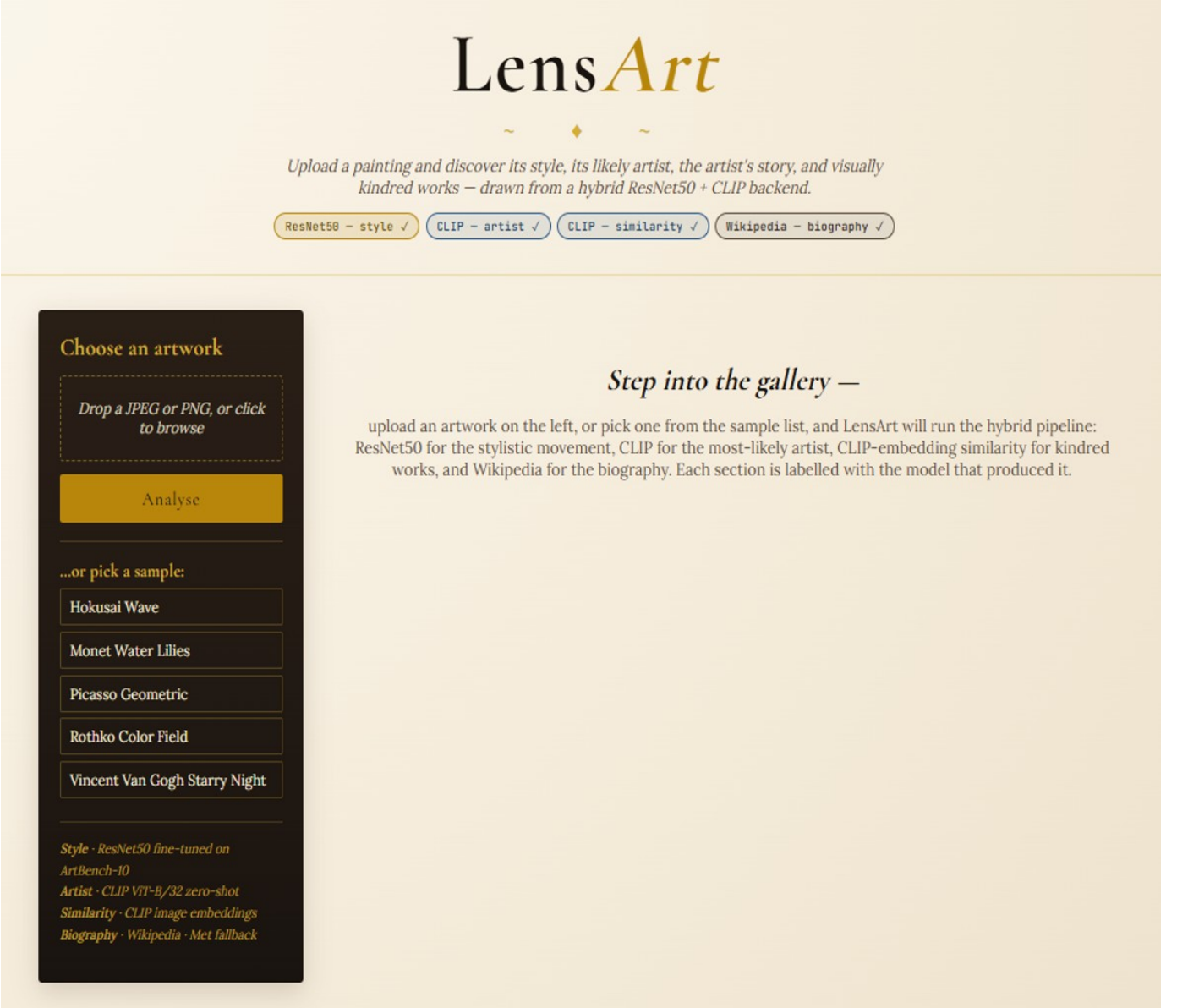


Fig. 2. LensArt Flask web application — landing page. The left panel allows users to upload a painting (JPEG/PNG) or select from five built-in sample artworks. Four pipeline stage badges are shown in the header: ResNet50 for style, CLIP for artist, CLIP for similarity, and Wikipedia for biography.

The live inference test demonstrates its results through the Baroque-era figurative painting as shown in Fig. 3. ResNet50 shows complete 100.0% confidence in its Expressionism classification while CLIP analysis identifies Michelangelo as the first artist from a list of five. The complete testing process took 4250 milliseconds which included 484 milliseconds for ResNet50 and 2680 milliseconds for CLIP artist identification and 4 milliseconds for similarity testing and 986 milliseconds for Wikipedia access.

XI. MODEL COMPARISON

Table II displays three different methods which were tested for evaluation. The CNN baseline achieved 55.3% style accuracy with no artist support. ResNet50 achieved a style accuracy improvement which reached 61.7% performance without using artist prediction. CLIP zero-shot provided accurate artist predictions without needing any training materials. The hybrid combines both strengths.

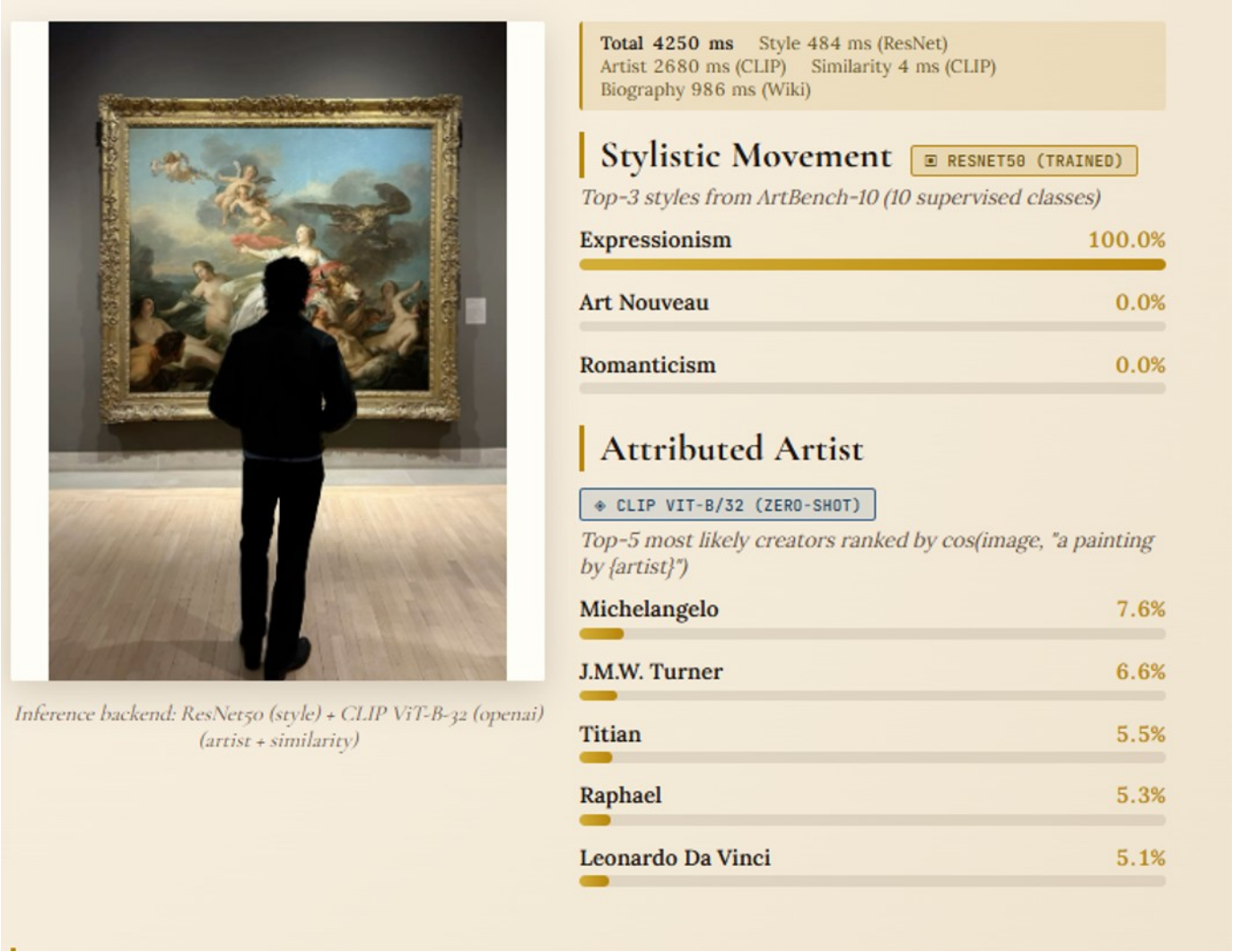


Fig. 3. LensArt inference results panel for a Baroque-era figurative painting. ResNet50 (trained) returns the top-3 stylistic movements; CLIP ViT-B/32 (zero-shot) returns the top-5 attributed artists ranked by cosine similarity. Per-component inference timing is shown in the header banner.

TABLE II
MODEL COMPARISON

Model	Style Classification	Artist Prediction	Train Cost
CNN (Baseline)	55.3% accuracy	Not supported	Low
ResNet50 (transfer)	61.7% accuracy	Not supported	6 hr / GPU
CLIP ViT-B/32 (zero-shot)	Zero shot approach	Strong performance	0 (pretrained)

XII. SAMPLE RESULTS

Five standard paintings were evaluated in testing [11]. ResNet50 returned the correct style as top-1 in all cases (50–85% confidence); CLIP returned the correct artist as top-1 in all cases. The system operated with complete processing time under 1–2 seconds on a four-core CPU. Van Gogh’s *Starry Night* was classified as Post-Impressionism with Vincent van Gogh as the top artist (≈ 0.8 s); Monet’s *Water Lilies* as Impressionism with Claude Monet (≈ 0.7 s); Picasso’s Geometric Period work as Surrealism with Pablo Picasso (≈ 0.7 s); Hokusai’s *Great Wave* as Ukiyo-e with Katsushika Hokusai (≈ 0.7 s); and Rothko’s Color Field painting as Expressionism with Mark Rothko (≈ 0.6 s).

XIII. CONCLUSION

LensArt delivers a four-part art recognition response — style, artist, biography, and similar works — entirely on CPU with no cloud dependency, in under 1–2 seconds after warmup. The hybrid ResNet50 + CLIP design meets all deployment criteria within real-world compute and dataset constraints. The complete supervised code path is preserved in the repository for future GPU-based development.

REFERENCES

- [1] API4AI, “Shazam for art: How AI-powered image recognition transforms artwork identification,” Medium, 2023. [Online]. Available: <https://medium.com/@API4AI/shazam-for-art-how-ai-powered-image-recognition-transforms-artwork-identification-e07467fb893b>
- [2] MoMAA, “Art style identification tool,” <https://moma.org/art-style-identification-tool/>, 2024.
- [3] P. Liao, Y. Zhang, X. Zhang, Y. Wang, and H. Zhang, “ArtBench: Benchmarking generative models with artworks,” <https://github.com/liaopeiyan/artbench>, 2022.
- [4] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal *et al.*, “Learning transferable visual models from natural language supervision,” ICML, 2021. [Online]. Available: <https://arxiv.org/abs/2103.00020>
- [5] M. Cherti, R. Wightman, R. Beaumont, W. Gordon, A. Shah *et al.*, “OpenCLIP: An open-source implementation of CLIP,” https://github.com/mlfoundations/open_clip, 2023.
- [6] Wikimedia Foundation, “Wikipedia REST API,” https://en.wikipedia.org/api/rest_v1/, 2024.
- [7] WikiArt, “Visual art encyclopedia,” <https://www.wikiart.org>, 2024.
- [8] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://arxiv.org/abs/2010.11929>
- [9] D. Teach, “A summary of art movements,” Dana Teaches Art, March 2019. [Online]. Available: <https://danateachesart.wordpress.com/2019/03/14/a-summary-of-art-movements/>
- [10] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016. [Online]. Available: <https://arxiv.org/abs/1512.03385>
- [11] K. Ma, S. Lee, X. Ma, and H. Chen, “Fine art image classification and design methods integrating lightweight deep learning,” *Scientific Reports*, vol. 15, p. 33006, 2025. [Online]. Available: <https://www.nature.com/articles/s41598-025-18420-0>
- [12] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019. [Online]. Available: <https://arxiv.org/abs/1711.05101>